CineVerse

An IMDb/Netflix-Inspired Movie Database System

ISE 503: Data Management - Spring 2026

Aditya Kamat, 116729750

Suhas Gangireddy, 116464631

Manasa Kumari, 116553874

27 Tables • 31,426 Rows • 10 Complex Queries • 9 App Screens

1. Project Overview

CineVerse is a relational movie database system that unifies movie metadata, cast/crew relationships, user ratings, reviews, streaming availability, awards, and personalized recommendations into a single PostgreSQL database. It is inspired by platforms like IMDb and Netflix but adds explainable mood-based recommendations, group-watch planning, and industry analytics.

The system uses real-world data from IMDb Non-Commercial Datasets (movies, people, credits, ratings) and MovieLens (user ratings, tags). The database contains 27 tables (14 entities + 13 bridge tables) with 31,426 rows, accessed through a React frontend and FastAPI backend.

Team Contributions

Team Member	Contributions
Aditya Kamat	Schema design (27 tables), ER model, constraints, bridge table strategy, relational mapping, queries Q1, Q2, Q5, Q8
Suhas Gangireddy	ETL pipeline (IMDb + MovieLens), data cleaning, validation, indexes, load scripts, queries Q3, Q4, Q7, Q10
Manasa Kumari	FastAPI backend (24 endpoints), React frontend (9 pages), deployment, demo flow, queries Q6, Q9

2. Database Design

The schema is organized into 6 logical layers: Core (movies, people, genres), User (ratings, reviews), Discovery (tags, recommendations), Streaming (platforms, availability), Industry (awards, studios, distributors), and Social (watchlists, group watch). All many-to-many relationships are resolved through bridge tables with composite primary keys.

Schema Summary

Layer	Entities	Bridges	Total
Core	Movie, Person, RoleType, Genre	MovieCredit, MovieGenre	6
User	UserProfile, Review	UserRating, ExternalRating	4
Discovery	Tag, RecommendationLog	MovieTagRelevance, UserPreferenceTag	4
Streaming	StreamingPlatform	MovieAvailability	2
Industry	Award, ProductionCompany, Distributor	MovieAward, MovieProductionCompany, MovieDistributor	6
Social	Watchlist, WatchParty	WatchlistItem, WatchPartyMember, WatchPartySuggestion	5
Total	14 entities	13 bridges	27

3. EER Diagrams

EER diagrams are attached and shown in the PPT.

4. Constraints Summary

Constraint Type	Count	Examples
Surrogate Primary Keys	15	movie_id SERIAL, person_id SERIAL
Composite Primary Keys	11	(user_id, movie_id), (movie_id, platform_id, region, access_type)
Foreign Keys	40+	MovieCredit.movie_id → Movie.movie_id
UNIQUE	11	Movie.imdb_id, UserProfile.username, Genre.genre_name
CHECK	18	rating BETWEEN 0 AND 10, runtime > 0, access_type IN (...)
NOT NULL	20+	title, imdb_id, username, rating, review_text
ON DELETE CASCADE	8	MovieGenre, UserRating, WatchlistItem
ON DELETE SET NULL	2	MovieAward.person_id, WatchPartySuggestion.suggested_by

Key Design Decisions

1. Unified Person table — one person can be actor, director, producer, and writer. Roles are tracked through MovieCredit + RoleType, not separate tables.

2. MovieAvailability uses a 4-column composite PK (movie_id, platform_id, region, access_type) so the same film on the same platform can be both subscription and rent.

3. MovieAward.person_id uses ON DELETE SET NULL — if a person is deleted, the award record survives with a nullified link.

5. SQL Queries (10 Complex Queries) [Output screenshots are present in PPT]

Q01: Top-Rated Movies by Genre

What's the best Sci-Fi film? Drama? Action? Rank the top 3 movies within each genre, with a 5-rating floor for statistical reliability. Uses RANK() window function and HAVING.

```
-- top 3 rated movies per genre, requires at least 5 user ratings to qualify
SELECT genre_name, title, release_year, avg_user_rating, num_ratings, genre_rank
FROM (
    SELECT
        g.genre_name,
        m.title,
```

```
        m.release_year,
        ROUND(AVG(ur.rating)::numeric, 2) AS avg_user_rating,
        COUNT(ur.user_id) AS num_ratings,
        RANK() OVER (PARTITION BY g.genre_name ORDER BY AVG(ur.rating) DESC) AS genre_rank
    FROM Movie m
    JOIN MovieGenre mg ON mg.movie_id = m.movie_id
    JOIN Genre g ON g.genre_id = mg.genre_id
    JOIN UserRating ur ON ur.movie_id = m.movie_id
    GROUP BY g.genre_name, m.movie_id, m.title, m.release_year
    HAVING COUNT(ur.user_id) >= 5
) ranked
WHERE genre_rank <= 3
ORDER BY genre_name, genre_rank;
```

Q02: Actor-Director Collaborations

Which actor-director duos keep reuniting? Find creative partnerships spanning 2+ films and list every movie they made together. Uses self-join on MovieCredit and STRING_AGG.

```
-- actors and directors who have worked together on 2+ films
SELECT
    p_actor.name AS actor_name,
    p_director.name AS director_name,
    COUNT(DISTINCT m.movie_id) AS movies_together,
    STRING_AGG(DISTINCT m.title, '; ' ORDER BY m.title) AS shared_movies
FROM MovieCredit mc_actor
JOIN RoleType rt_actor ON rt_actor.role_type_id = mc_actor.role_type_id
    AND rt_actor.role_name = 'Actor'
JOIN Person p_actor ON p_actor.person_id = mc_actor.person_id
JOIN MovieCredit mc_director ON mc_director.movie_id = mc_actor.movie_id
JOIN RoleType rt_director ON rt_director.role_type_id = mc_director.role_type_id
    AND rt_director.role_name = 'Director'
JOIN Person p_director ON p_director.person_id = mc_director.person_id
JOIN Movie m ON m.movie_id = mc_actor.movie_id
GROUP BY p_actor.name, p_director.name
HAVING COUNT(DISTINCT m.movie_id) >= 2
ORDER BY movies_together DESC, actor_name
LIMIT 30;
```

Q03: Audience vs Critic Gap

Which movies do audiences love but critics dismiss, and vice versa? Compare user ratings against IMDb scores. A gap > 1.0 flags a divergence. Uses CASE expression and AVG comparison.

```
-- movies where user ratings diverge significantly from IMDb critic scores
SELECT
    m.title,
    m.release_year,
    ROUND(AVG(ur.rating)::numeric, 2) AS avg_user_rating,
    er.score AS imdb_score,
    ROUND((AVG(ur.rating) - er.score)::numeric, 2) AS user_vs_critic_gap,
    COUNT(ur.user_id) AS num_user_ratings,
    CASE
        WHEN AVG(ur.rating) - er.score > 1.0 THEN 'Audience Favorite'
        WHEN er.score - AVG(ur.rating) > 1.0 THEN 'Critic Favorite'
        ELSE 'Consensus'
    END AS gap_category
FROM Movie m
JOIN UserRating ur ON ur.movie_id = m.movie_id
JOIN ExternalRating er ON er.movie_id = m.movie_id AND er.source_name = 'IMDb'
```

```
GROUP BY m.movie_id, m.title, m.release_year, er.score
HAVING COUNT(ur.user_id) >= 3
ORDER BY ABS(AVG(ur.rating) - er.score) DESC
LIMIT 25;
```

Q04: Mood-Based Recommendations

I loved Inception and Interstellar — what should I watch next, and why? Recommend movies based on mood tags from highly-rated films, excluding already-rated ones. Uses subquery and LEFT JOIN exclusion.

```
-- recommend movies to user 1 based on tags from their highly-rated movies,
-- excluding anything they've already rated
SELECT
    m.movie_id,
    m.title,
    m.release_year,
    m.imdb_rating,
    ROUND(SUM(mtr.relevance_score), 3) AS recommendation_score,
    STRING_AGG(DISTINCT t.tag_name, ', ' ORDER BY t.tag_name) AS matching_tags,
    COUNT(DISTINCT t.tag_id) AS tag_match_count
FROM MovieTagRelevance mtr
JOIN Tag t ON t.tag_id = mtr.tag_id
JOIN Movie m ON m.movie_id = mtr.movie_id
LEFT JOIN UserRating ur ON ur.movie_id = m.movie_id AND ur.user_id = 1
WHERE mtr.relevance_score >= 0.5
  AND ur.user_id IS NULL  -- exclude already-rated movies
  AND t.tag_name IN (
      -- get tags from movies user 1 rated highly
      SELECT DISTINCT t2.tag_name
      FROM UserRating ur2
      JOIN MovieTagRelevance mtr2 ON mtr2.movie_id = ur2.movie_id
      JOIN Tag t2 ON t2.tag_id = mtr2.tag_id
      WHERE ur2.user_id = 1 AND ur2.rating >= 8.0 AND mtr2.relevance_score >= 0.6
  )
GROUP BY m.movie_id, m.title, m.release_year, m.imdb_rating
ORDER BY recommendation_score DESC
LIMIT 15;
```

Q05: Hidden Gems

What great movies am I missing because nobody talks about them? Find films rated 7.5+ with below-median IMDb vote counts. Uses PERCENTILE_CONT and LOG weighted scoring.

```
-- high-rated movies with below-median vote counts (hidden gems)
SELECT
    m.title,
    m.release_year,
    ROUND(AVG(ur.rating)::numeric, 2) AS avg_user_rating,
    COUNT(ur.user_id) AS num_ratings,
    m.imdb_votes,
    m.imdb_rating,
    ROUND((AVG(ur.rating) * LOG(COUNT(ur.user_id) + 1))::numeric, 2) AS weighted_score
FROM Movie m
JOIN UserRating ur ON ur.movie_id = m.movie_id
WHERE m.imdb_votes < (SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY imdb_votes) FROM Movie)
GROUP BY m.movie_id, m.title, m.release_year, m.imdb_votes, m.imdb_rating
HAVING COUNT(ur.user_id) >= 3 AND AVG(ur.rating) >= 7.5
ORDER BY avg_user_rating DESC, num_ratings DESC
LIMIT 20;
```

Q06: Most Versatile Actors

Who has the most range — comedy, horror, drama, action all in one career? Rank actors by distinct genres across 3+ films. Uses COUNT DISTINCT and DENSE_RANK.

```
-- actors who have appeared in the most distinct genres
SELECT
    p.name AS actor_name,
    COUNT(DISTINCT g.genre_id) AS genre_count,
    STRING_AGG(DISTINCT g.genre_name, ', ' ORDER BY g.genre_name) AS genres,
    COUNT(DISTINCT mc.movie_id) AS total_movies,
    DENSE_RANK() OVER (ORDER BY COUNT(DISTINCT g.genre_id) DESC) AS versatility_rank
FROM Person p
JOIN MovieCredit mc ON mc.person_id = p.person_id
JOIN RoleType rt ON rt.role_type_id = mc.role_type_id AND rt.role_name = 'Actor'
JOIN MovieGenre mg ON mg.movie_id = mc.movie_id
JOIN Genre g ON g.genre_id = mg.genre_id
GROUP BY p.person_id, p.name
HAVING COUNT(DISTINCT mc.movie_id) >= 3
ORDER BY genre_count DESC, total_movies DESC
LIMIT 20;
```

Q07: Top Directors by Rating

Which directors consistently make the highest-rated films? Rank directors with 3+ films by average IMDb rating and total audience reach. Uses aggregate functions and STRING_AGG.

```
-- directors ranked by average IMDb rating and total audience reach
-- (uses imdb_rating and imdb_votes since budget/revenue require TMDb enrichment)
SELECT
    p.name AS director_name,
    COUNT(DISTINCT m.movie_id) AS num_films,
    ROUND(AVG(m.imdb_rating)::numeric, 2) AS avg_imdb_rating,
    SUM(m.imdb_votes) AS total_votes,
    ROUND(AVG(ur.rating)::numeric, 2) AS avg_user_rating,
    COUNT(DISTINCT ur.user_id) AS unique_raters,
    STRING_AGG(DISTINCT m.title, '; ' ORDER BY m.title) AS films
FROM Person p
JOIN MovieCredit mc ON mc.person_id = p.person_id
JOIN RoleType rt ON rt.role_type_id = mc.role_type_id AND rt.role_name = 'Director'
JOIN Movie m ON m.movie_id = mc.movie_id
LEFT JOIN UserRating ur ON ur.movie_id = m.movie_id
WHERE m.imdb_rating IS NOT NULL
GROUP BY p.person_id, p.name
HAVING COUNT(DISTINCT m.movie_id) >= 3
ORDER BY avg_imdb_rating DESC, total_votes DESC
LIMIT 20;
```

Q08: Streaming Availability Search

What Action or Sci-Fi movies can I watch on Netflix, Disney+, or Max right now? Filter by genre, platform, and subscription access. Uses multi-table JOIN with WHERE filters.

```
-- find action/sci-fi/thriller movies on Netflix, Disney+, or Max via subscription in the US
SELECT DISTINCT
    m.title,
    m.release_year,
    m.imdb_rating,
    g.genre_name,
    sp.name AS platform_name,
```

```
    ma.access_type,
    ma.region
FROM Movie m
JOIN MovieAvailability ma ON ma.movie_id = m.movie_id
JOIN StreamingPlatform sp ON sp.platform_id = ma.platform_id
JOIN MovieGenre mg ON mg.movie_id = m.movie_id
JOIN Genre g ON g.genre_id = mg.genre_id
WHERE g.genre_name IN ('Action', 'Sci-Fi', 'Thriller')
  AND sp.name IN ('Netflix', 'Disney+', 'Max (HBO)')
  AND ma.region = 'US'
  AND ma.access_type = 'subscription'
ORDER BY m.imdb_rating DESC NULLS LAST, m.title
LIMIT 30;
```

Q09: Award Winners on Streaming

I want to watch something Oscar-worthy tonight — what's streaming? Cross-reference award wins with streaming availability. Uses 5-table JOIN with STRING_AGG for awards and platforms.

```
-- award-winning movies currently available to stream in the US
SELECT
    m.title,
    m.release_year,
    m.imdb_rating,
    STRING_AGG(DISTINCT a.award_name || ' - ' || a.category, '; ') AS awards_won,
    COUNT(DISTINCT ma2.award_id) AS total_wins,
    STRING_AGG(DISTINCT sp.name, ', ') AS available_on
FROM Movie m
JOIN MovieAward ma2 ON ma2.movie_id = m.movie_id AND ma2.result = 'won'
JOIN Award a ON a.award_id = ma2.award_id
JOIN MovieAvailability ma ON ma.movie_id = m.movie_id
JOIN StreamingPlatform sp ON sp.platform_id = ma.platform_id
WHERE ma.region = 'US'
  AND ma.access_type = 'subscription'
  AND (ma.end_date IS NULL OR ma.end_date >= CURRENT_DATE)
GROUP BY m.movie_id, m.title, m.release_year, m.imdb_rating
HAVING COUNT(DISTINCT ma2.award_id) >= 1
ORDER BY total_wins DESC, m.imdb_rating DESC NULLS LAST
LIMIT 20;
```

Q10: Group Watch Recommendation

My friends and I want a movie night — what hasn't any of us seen yet? Exclude every movie any party member has rated, rank by community score. Uses NOT EXISTS anti-join pattern.

```
-- movies no watch party member has seen, ranked by community rating
SELECT
    m.title,
    m.release_year,
    m.imdb_rating,
    ROUND(AVG(ur.rating)::numeric, 2) AS avg_user_rating,
    COUNT(DISTINCT ur.user_id) AS times_rated,
    STRING_AGG(DISTINCT g.genre_name, ', ' ORDER BY g.genre_name) AS genres
FROM Movie m
JOIN UserRating ur ON ur.movie_id = m.movie_id
JOIN MovieGenre mg ON mg.movie_id = m.movie_id
JOIN Genre g ON g.genre_id = mg.genre_id
WHERE NOT EXISTS (
    SELECT 1 FROM WatchPartyMember wpm
    JOIN UserRating seen ON seen.user_id = wpm.user_id AND seen.movie_id = m.movie_id
```

```
    WHERE wpm.party_id = 1
)
GROUP BY m.movie_id, m.title, m.release_year, m.imdb_rating
HAVING COUNT(DISTINCT ur.user_id) >= 3
ORDER BY avg_user_rating DESC
LIMIT 10;
```

6. Data Volume

Data sourced from IMDb Non-Commercial Datasets and MovieLens Latest Small.

Table	Rows	Source
Movie	500	IMDb (filtered, 1970+)
Person	5,166	IMDb name.basics
MovieCredit	10,021	IMDb title.principals
RoleType	4	Manual
Genre	20	IMDb genres
MovieGenre	1,342	Normalized links
UserProfile	200	MovieLens user IDs
UserRating	9,874	MovieLens (scaled 0-10)
ExternalRating	500	IMDb averageRating
Tag	200	MovieLens user tags
MovieTagRelevance	754	Tag frequency scores
MovieAvailability	995	Curated assignment
MovieAward	158	Wins and nominations
Total (all 27 tables)	31,426	Avg 1,309 per table